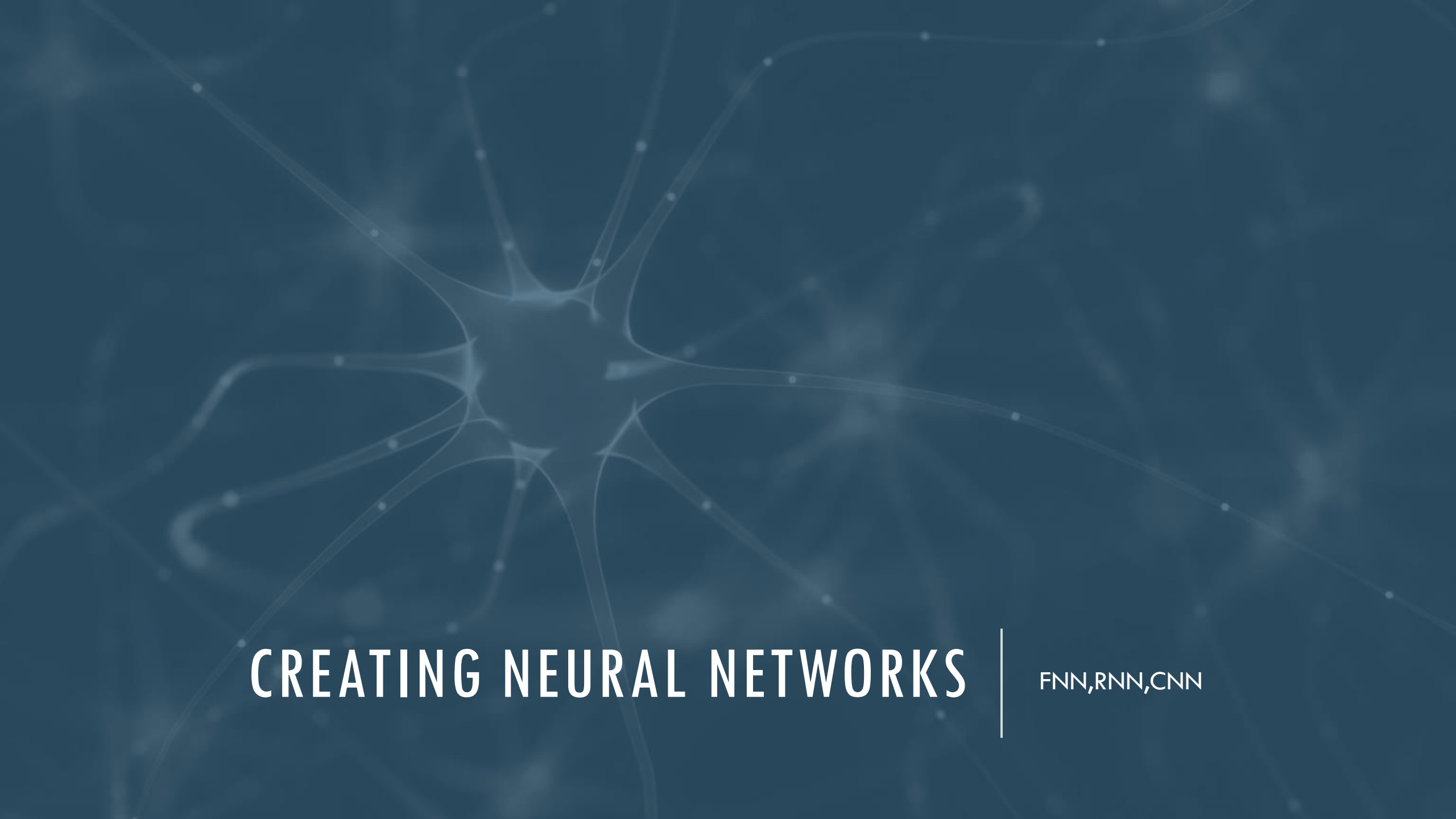




CREATING NEURAL NETWORKS

FNN,RNN,CNN

FEEDFORWARD NEURAL NETWORK (FNN)

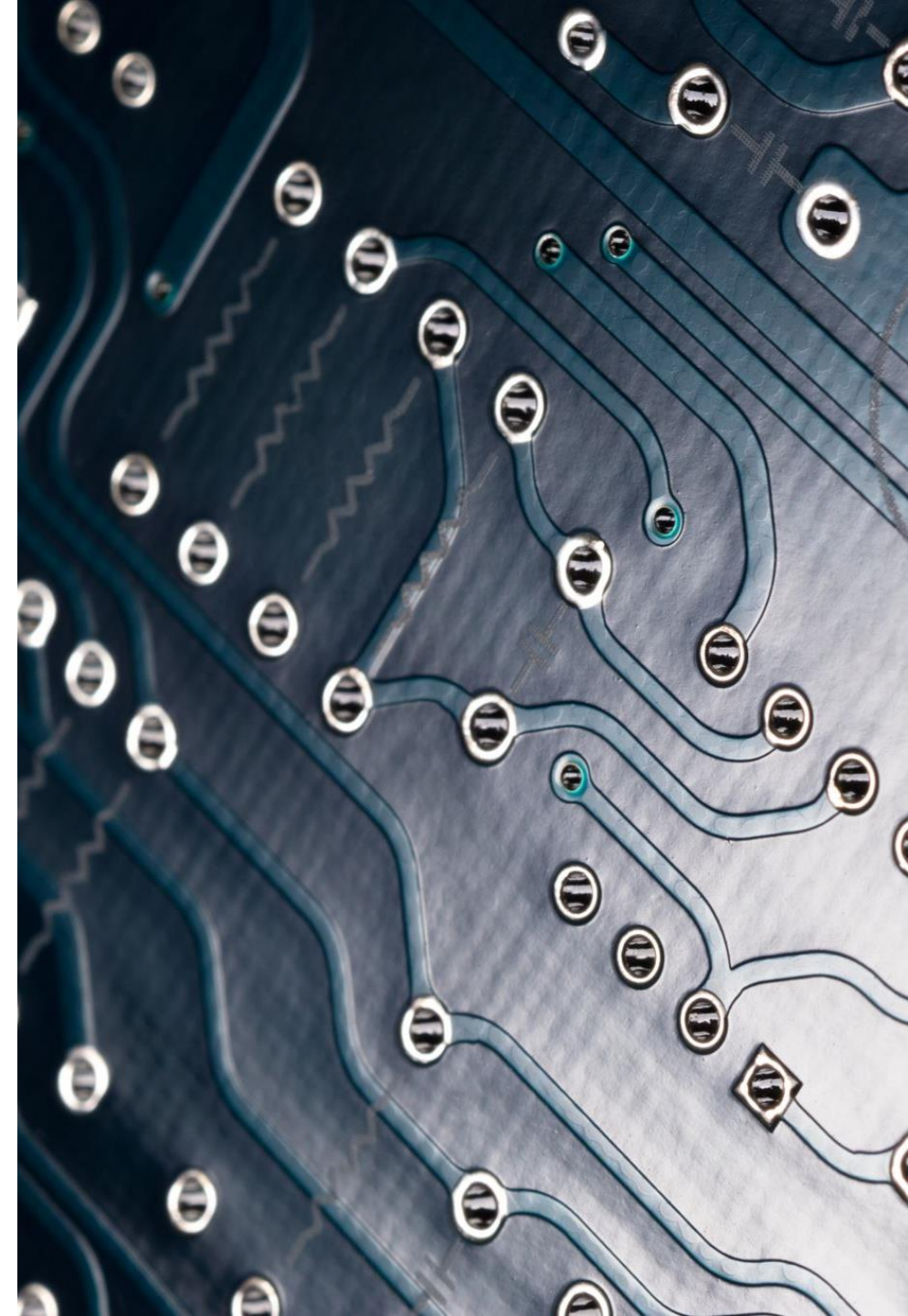
A Feedforward Neural Network (FNN) is one of the simplest types of artificial neural networks. A Feedforward Neural Network (FNN) is a type of artificial neural network where connections between the nodes do not form cycles.



KEY CHARACTERISTICS OF FEEDFORWARD NEURAL NETWORKS

1. Layered Structure:

1. **Input Layer:** The first layer that receives the input data.
2. **Hidden Layers:** One or more layers between the input and output layers where computations are performed. These layers are called "hidden" because they are not directly observable from the input or output.
3. **Output Layer:** The final layer that produces the output of the network.



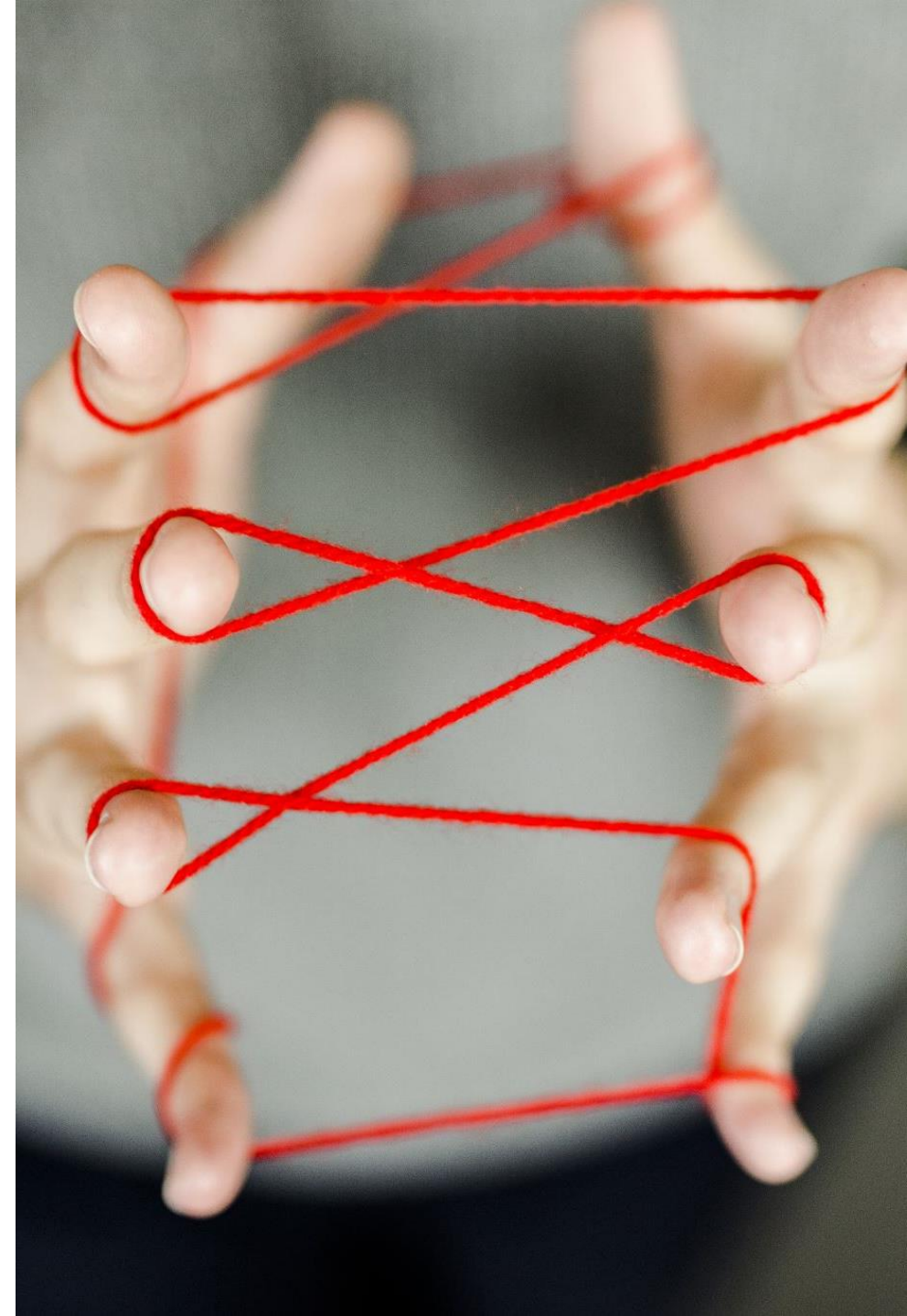


Unidirectional Flow:

1. **Forward Propagation:** Data flows in one direction, from the input layer through the hidden layers to the output layer. There are no cycles or loops in the network.

Activation Functions:

1. **Purpose:** Introduce non-linearity into the model, allowing it to learn complex patterns. Common activation functions include ReLU (Rectified Linear Unit), sigmoid, and tanh.

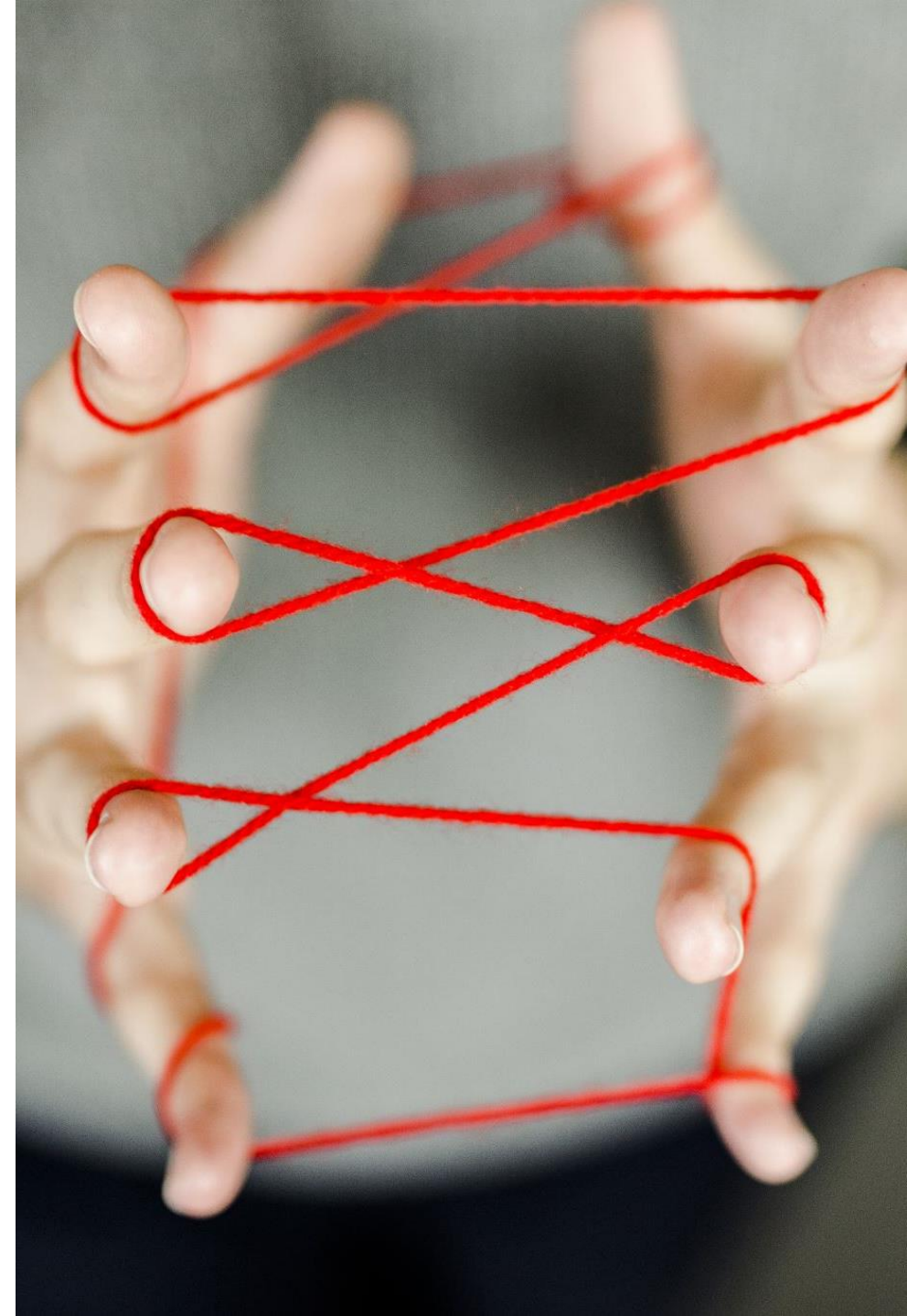


HOW FEEDFORWARD NEURAL NETWORKS WORK

- 1. Input Data:** The input data is fed into the input layer.
- 2. Weighted Sum:** Each neuron in a layer computes a weighted sum of its inputs.
- 3. Activation Function:** The weighted sum is passed through an activation function to introduce non-linearity.
- 4. Output Generation:** The process is repeated for each layer until the output layer produces the final result.

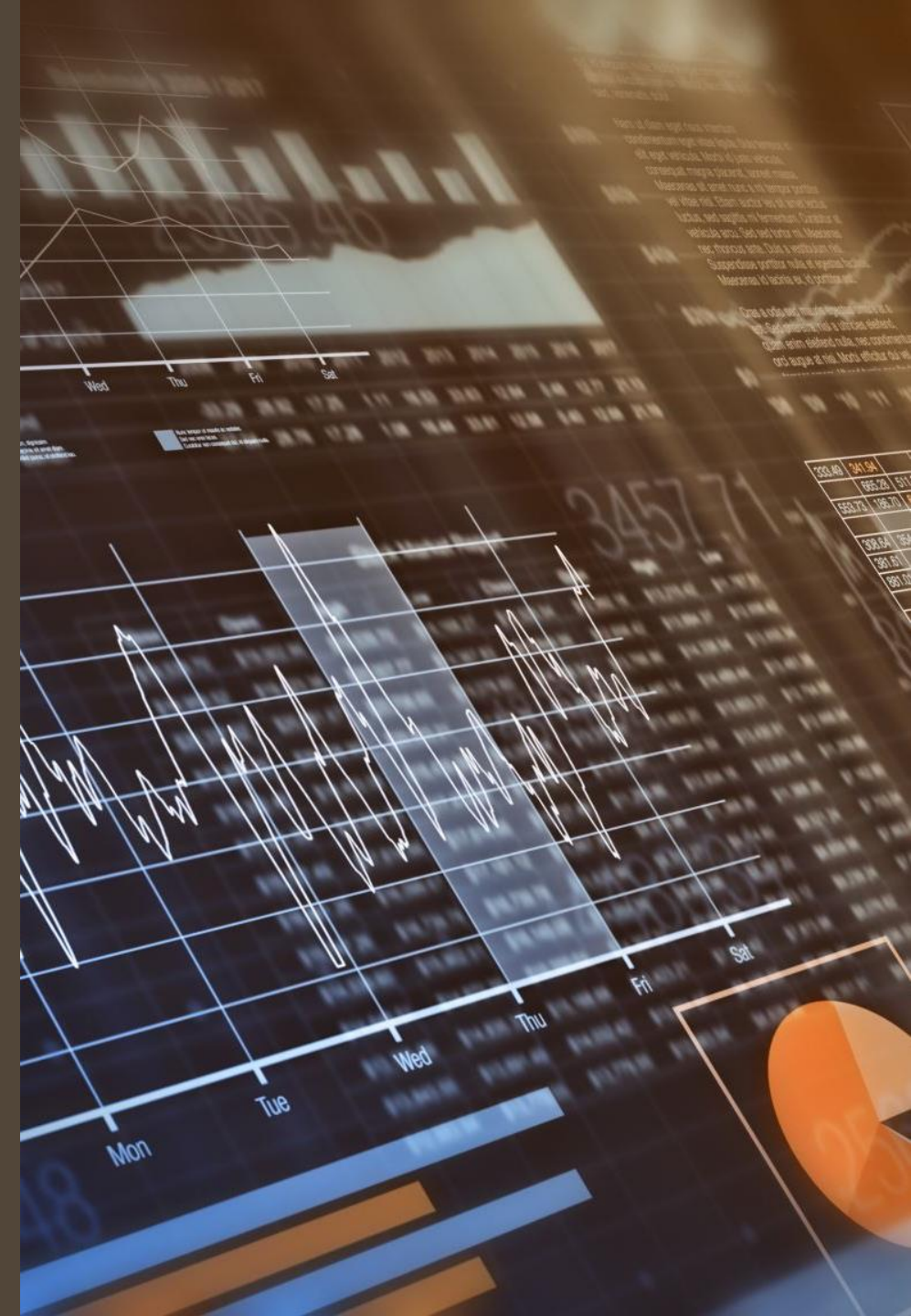
TRAINING FEEDFORWARD NEURAL NETWORKS

- **Loss Function:** A loss function measures the difference between the predicted output and the actual target values.
- **Backpropagation:** The network is trained using backpropagation, where the gradients of the loss function with respect to the weights are computed and used to update the weights to minimize the loss.



APPLICATIONS OF FEEDFORWARD NEURAL NETWORKS

- **Classification:** Tasks like image recognition, spam detection, and medical diagnosis.
- **Regression:** Predicting continuous values, such as house prices or stock prices.
- **Function Approximation:** Approximating complex functions in various scientific and engineering applications.



RECURRENT NEURAL NETWORK

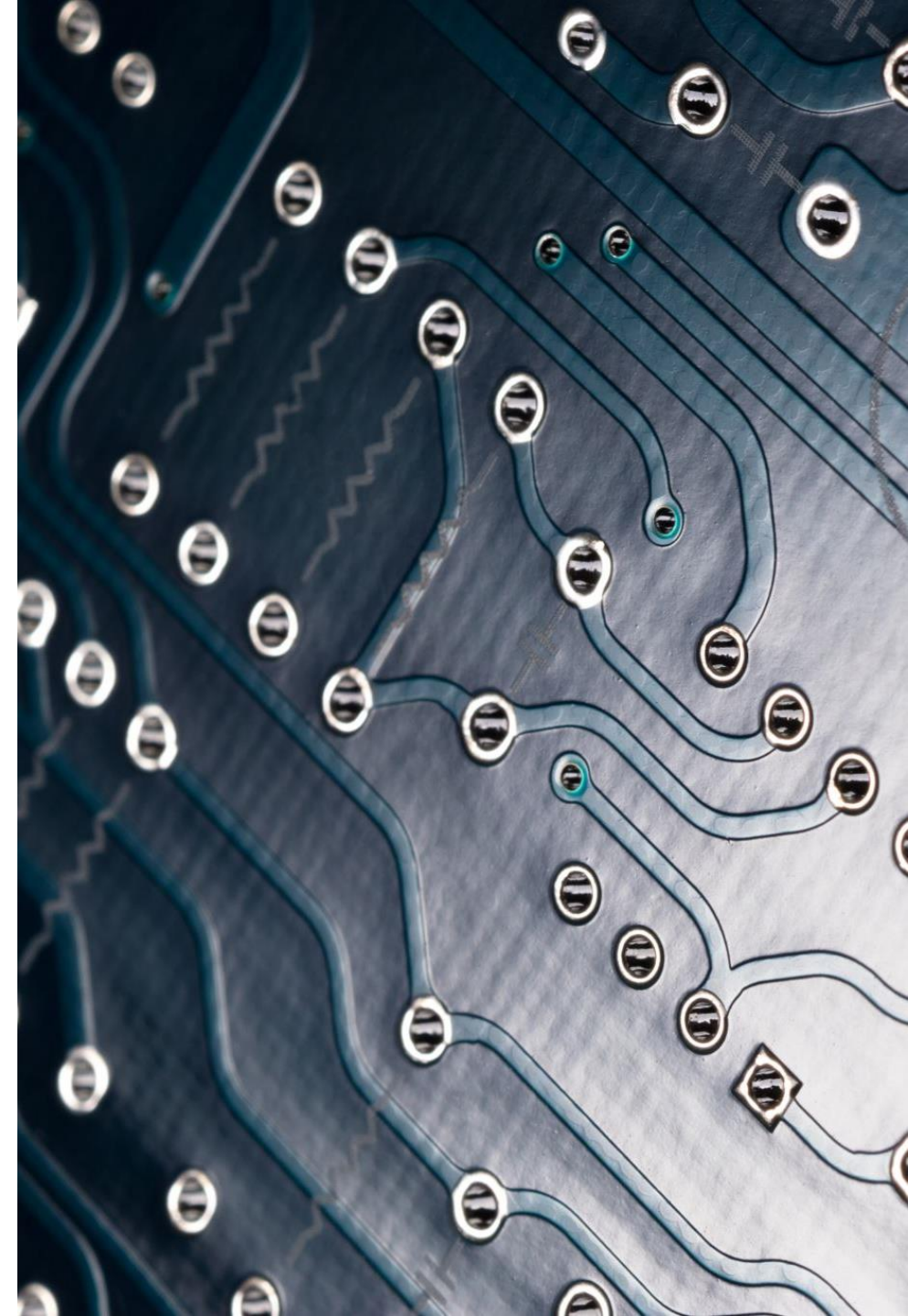
A Recurrent Neural Network (RNN) is a type of neural network designed to handle sequential data, where the order of the data points is important.



KEY COMPONENTS OF RNNs

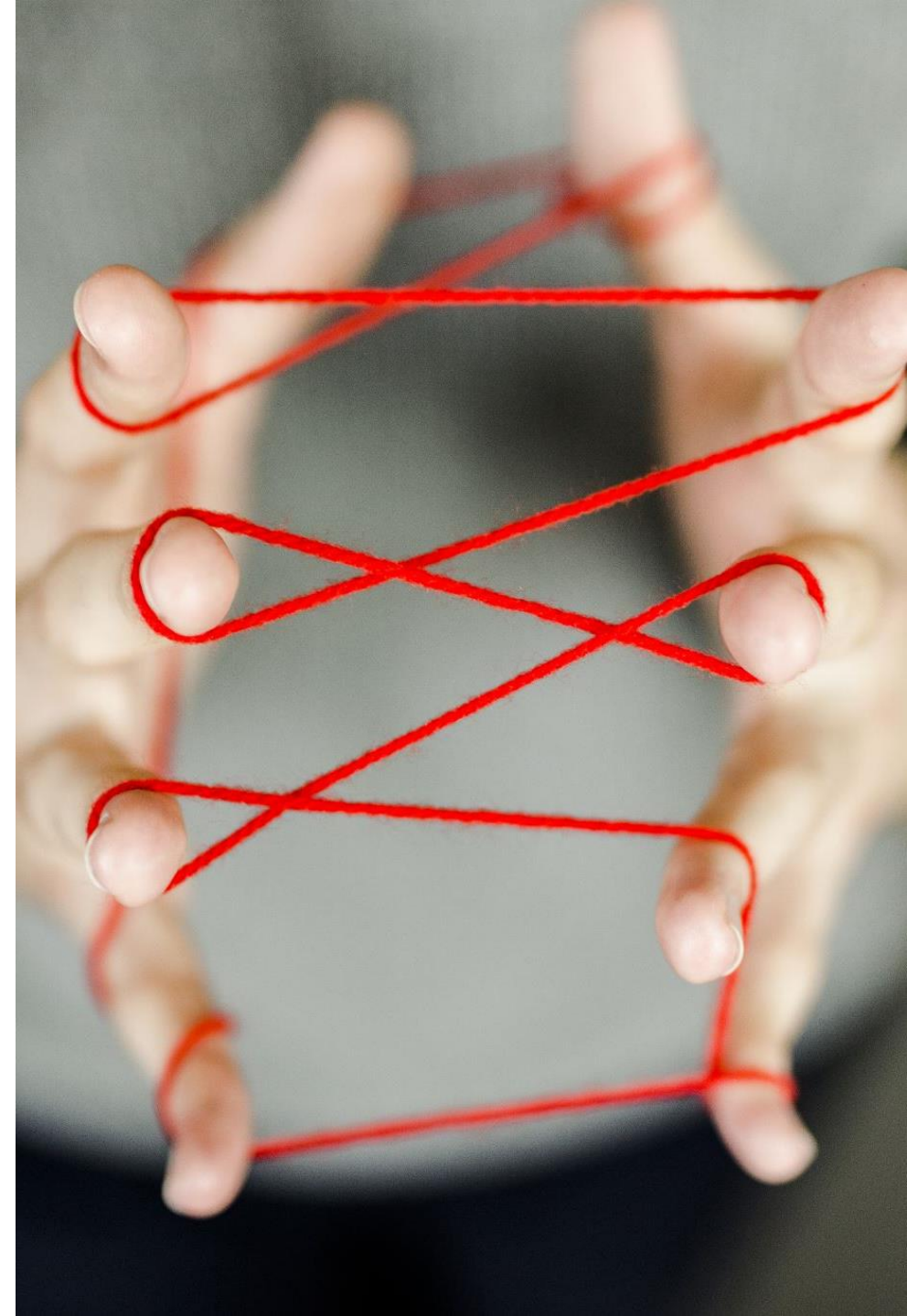
1. Recurrent Layers:

1. **Recurrent Connections:** Unlike traditional neural networks, RNNs have connections that loop back on themselves. This allows them to maintain a memory of previous inputs and use this information to influence the current output.
2. **Hidden State:** The hidden state is a dynamic memory that captures information from previous time steps. It is updated at each time step based on the current input and the previous hidden state.



Activation Functions:

1. **Purpose:** Introduce non-linearity into the model, allowing it to learn complex patterns. Common activation functions in RNNs include tanh and ReLU.



HOW RNNS WORK



Input Sequence: The input data is fed into the network one time step at a time.



Hidden State Update: At each time step, the hidden state is updated based on the current input and the previous hidden state.



Output Generation: The network generates an output at each time step, which can be used for tasks like sequence prediction or classification.



Backpropagation Through Time (BPTT): The network is trained using a variant of backpropagation called BPTT, which takes into account the dependencies across time steps.

APPLICATIONS OF RNNS



Natural Language Processing (NLP): Tasks like language modeling, text generation, machine translation, and sentiment analysis.



Speech Recognition: Converting spoken language into text.



Time Series Prediction: Forecasting future values based on past data, such as stock prices or weather patterns.



Sequence Classification: Classifying sequences of data, such as identifying the genre of a piece of music or detecting anomalies in sensor data.

VARIANTS OF RNNS

- **Long Short-Term Memory (LSTM):** A type of RNN designed to handle long-term dependencies by using special units called memory cells. LSTMs are effective at mitigating the vanishing gradient problem.
- **Gated Recurrent Unit (GRU):** A simplified version of LSTM that also addresses the vanishing gradient problem but with fewer parameters.

The vanishing gradient problem occurs when the gradients of the loss function with respect to the model parameters become very small during backpropagation. This makes it difficult for the network to learn and update the weights effectively.

CNN

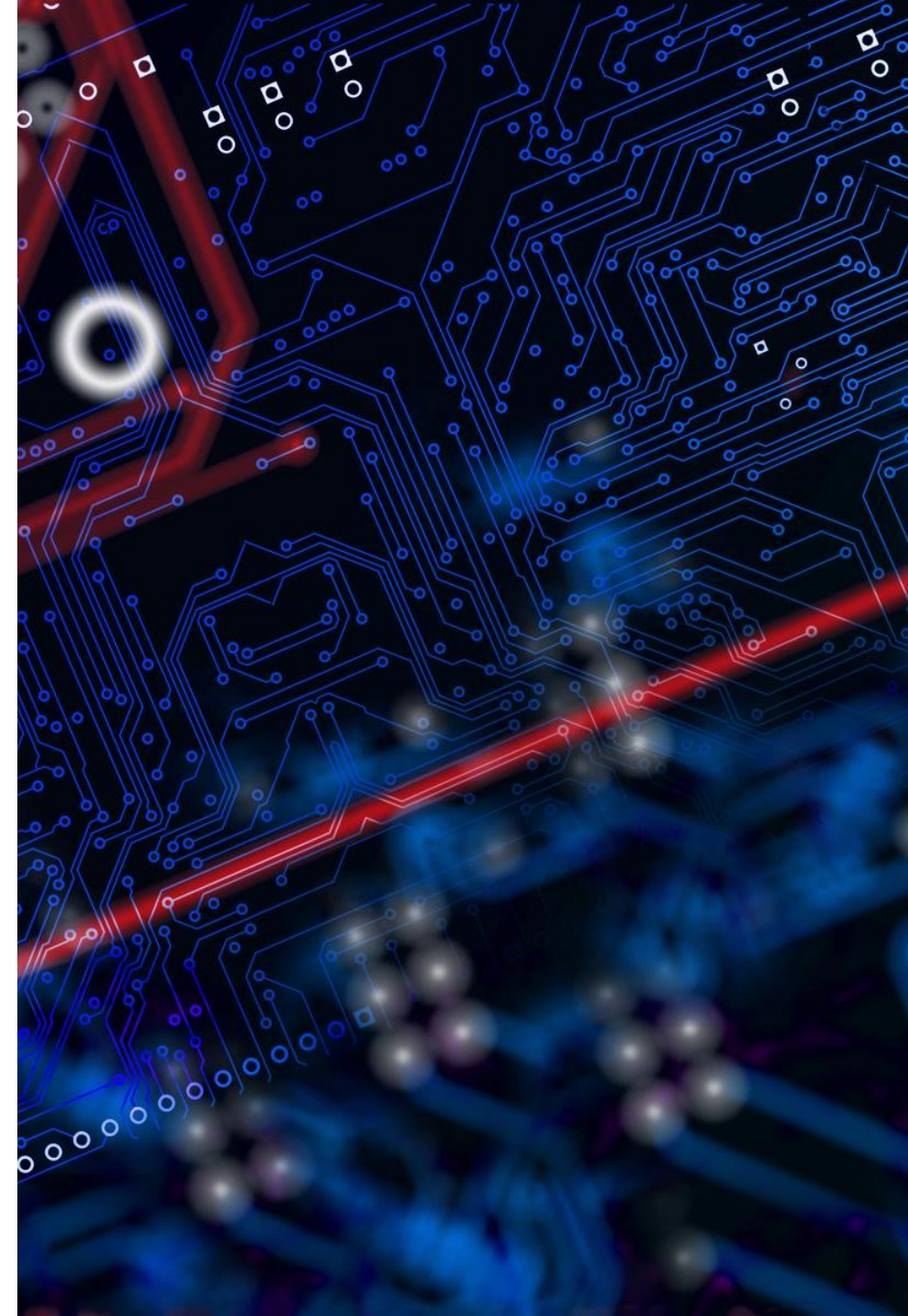
A Convolutional Neural Network (CNN) is a type of deep learning model specifically designed for processing structured grid data, such as images.



KEY COMPONENTS OF CNNs

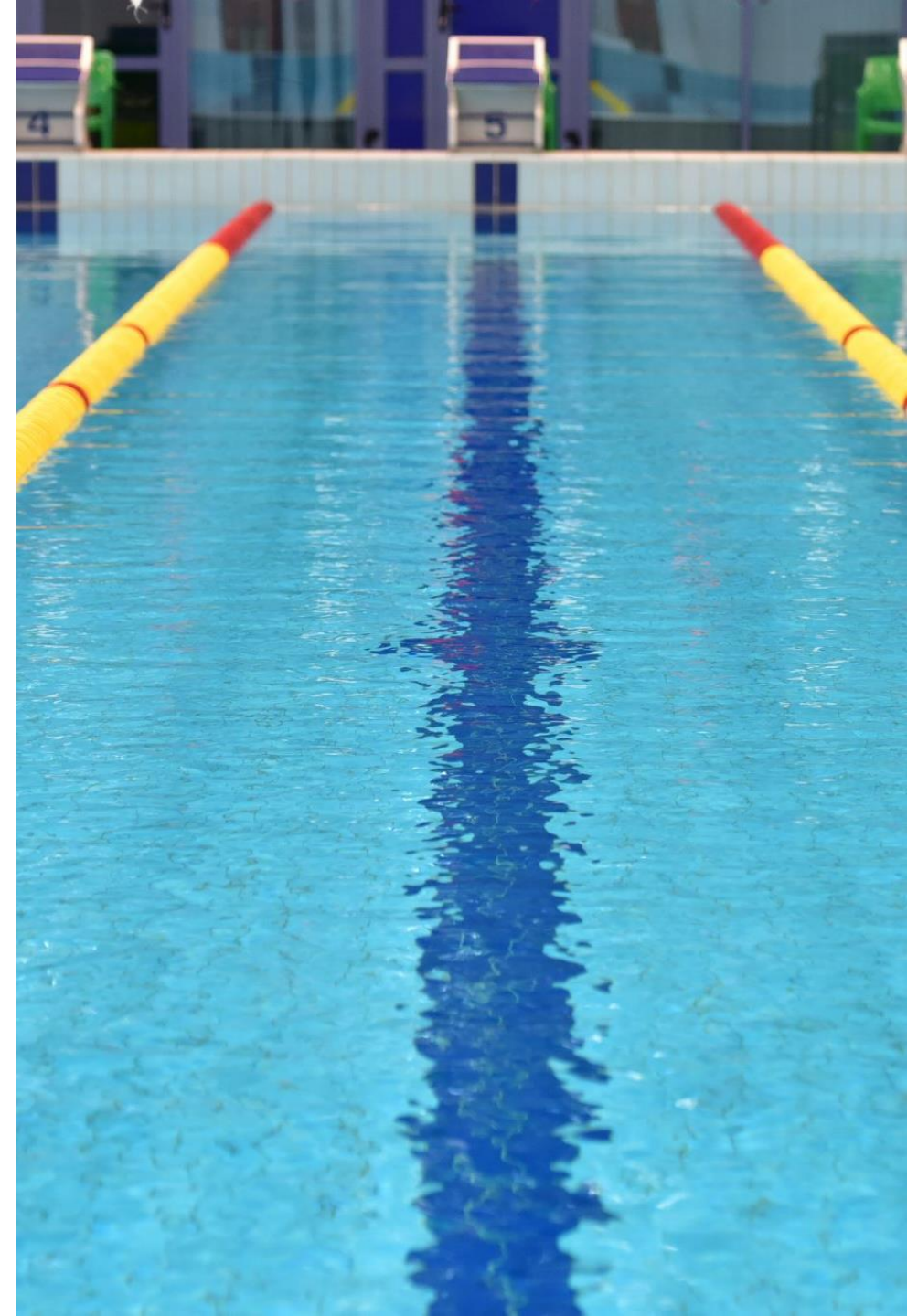
Convolutional Layers:

1. **Filters/Kernels:** Small matrices that slide over the input data to detect features like edges, textures, and patterns.
2. **Feature Maps:** The output of the convolution operation, highlighting the presence of specific features in the input data.

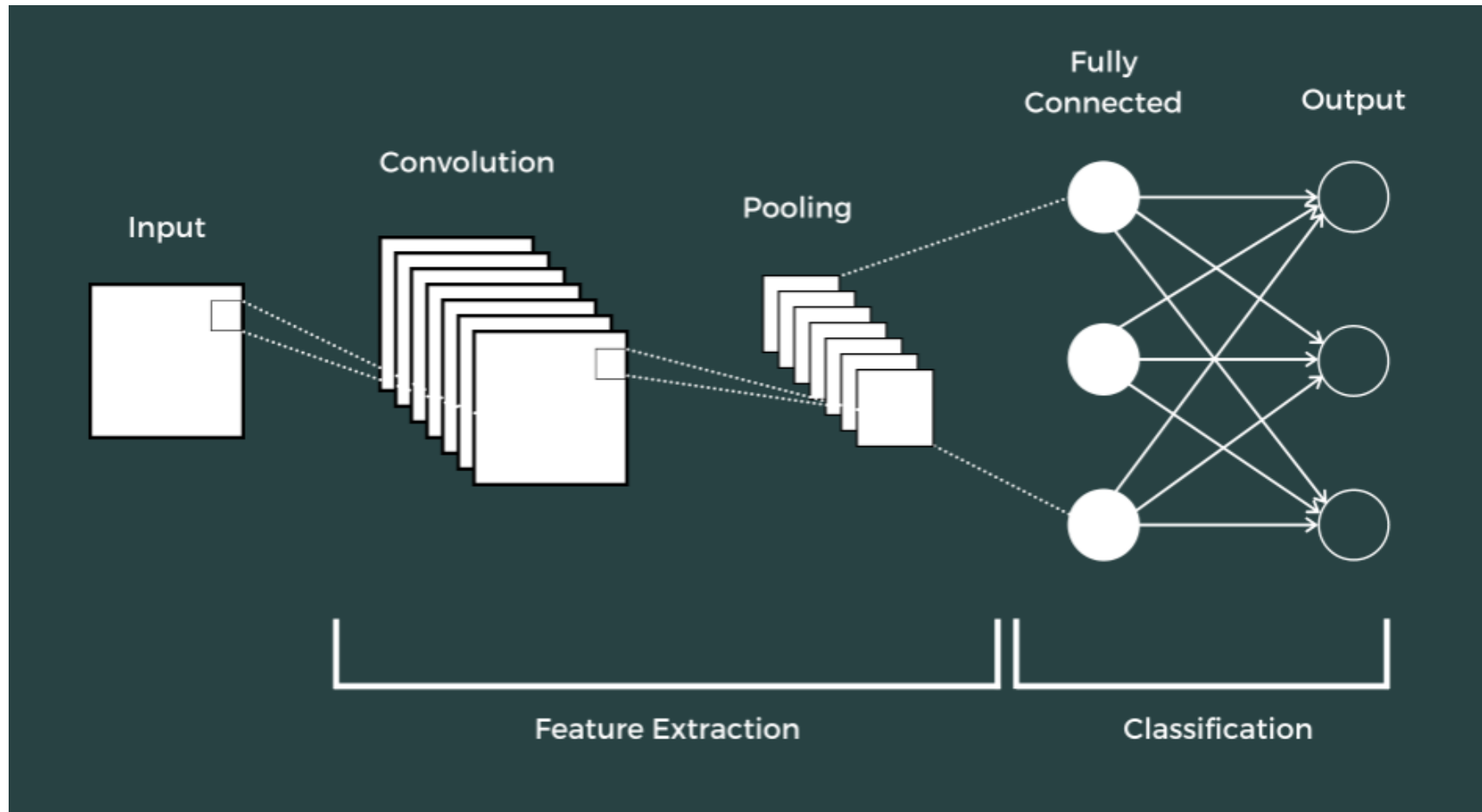


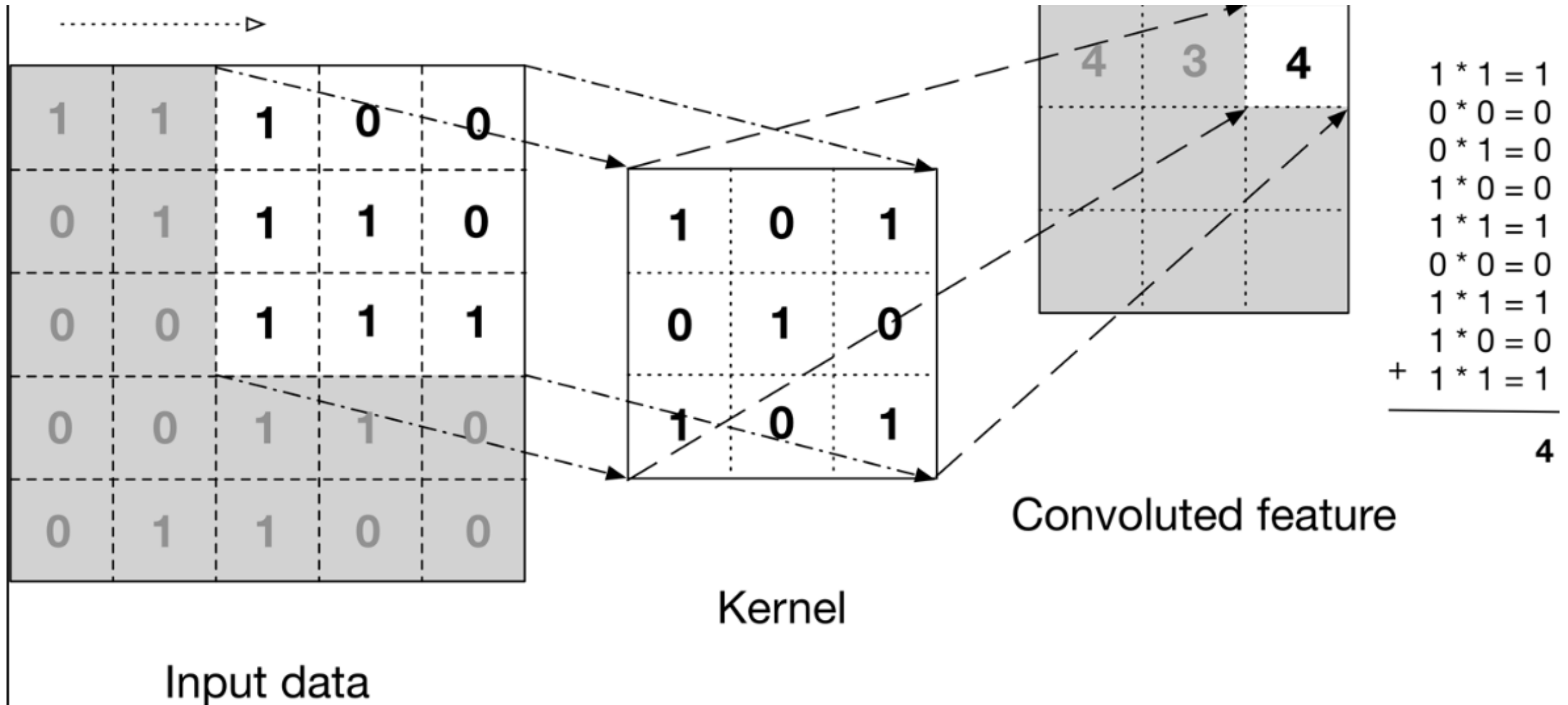
Pooling Layers:

1. **Purpose:** Reduce the spatial dimensions (width and height) of the feature maps, which helps in reducing the computational load and controlling overfitting.
2. **Types:** Common types include max pooling (selecting the maximum value) and average pooling (calculating the average value).



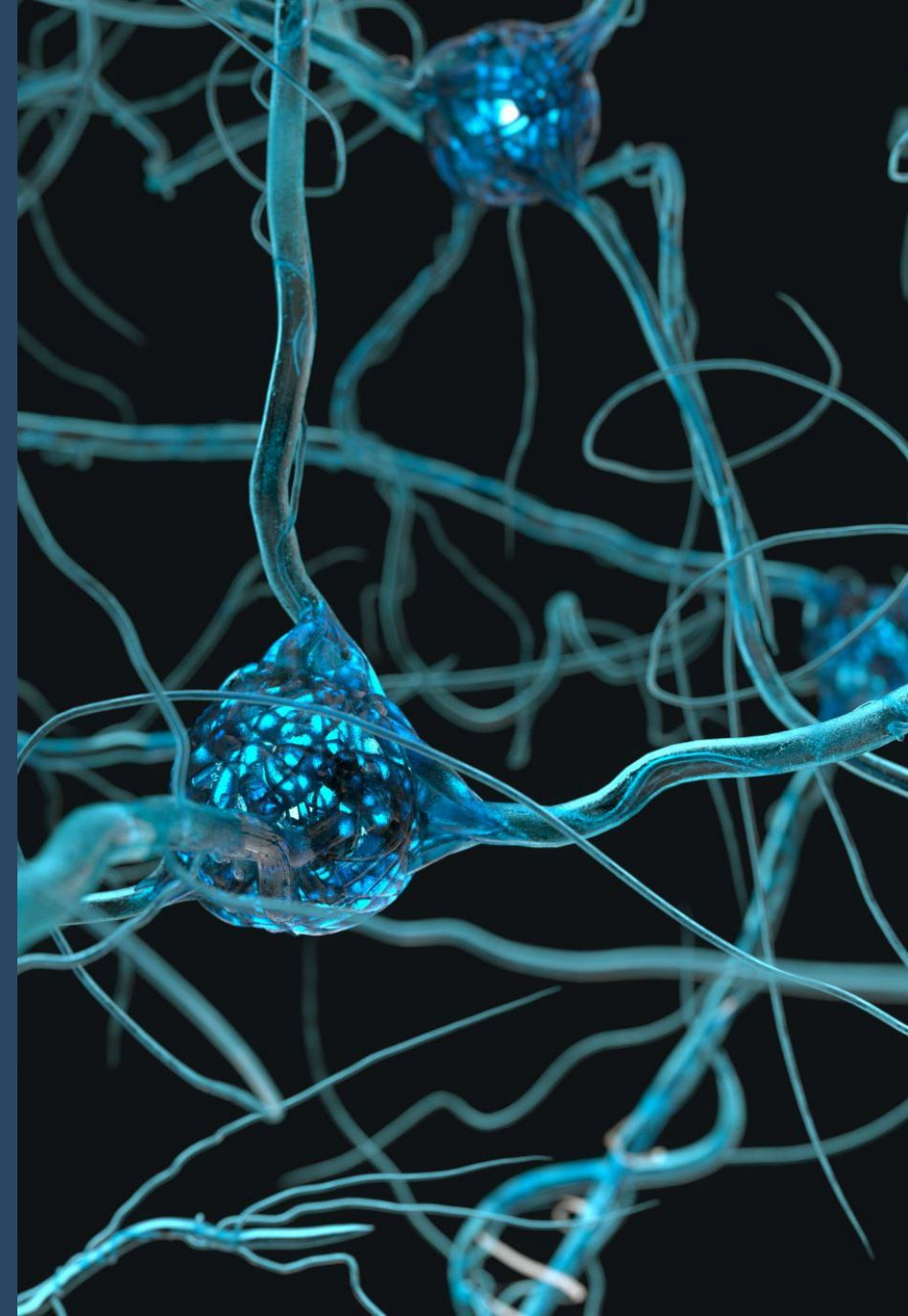
IMG SRC: [HTTPS://WWW.THECLICKREADER.COM/WP-CONTENT/UPLOADS/2020/07/CNN-ARCHITECTURE.PNG](https://www.theclickreader.com/wp-content/uploads/2020/07/CNN-ARCHITECTURE.PNG)





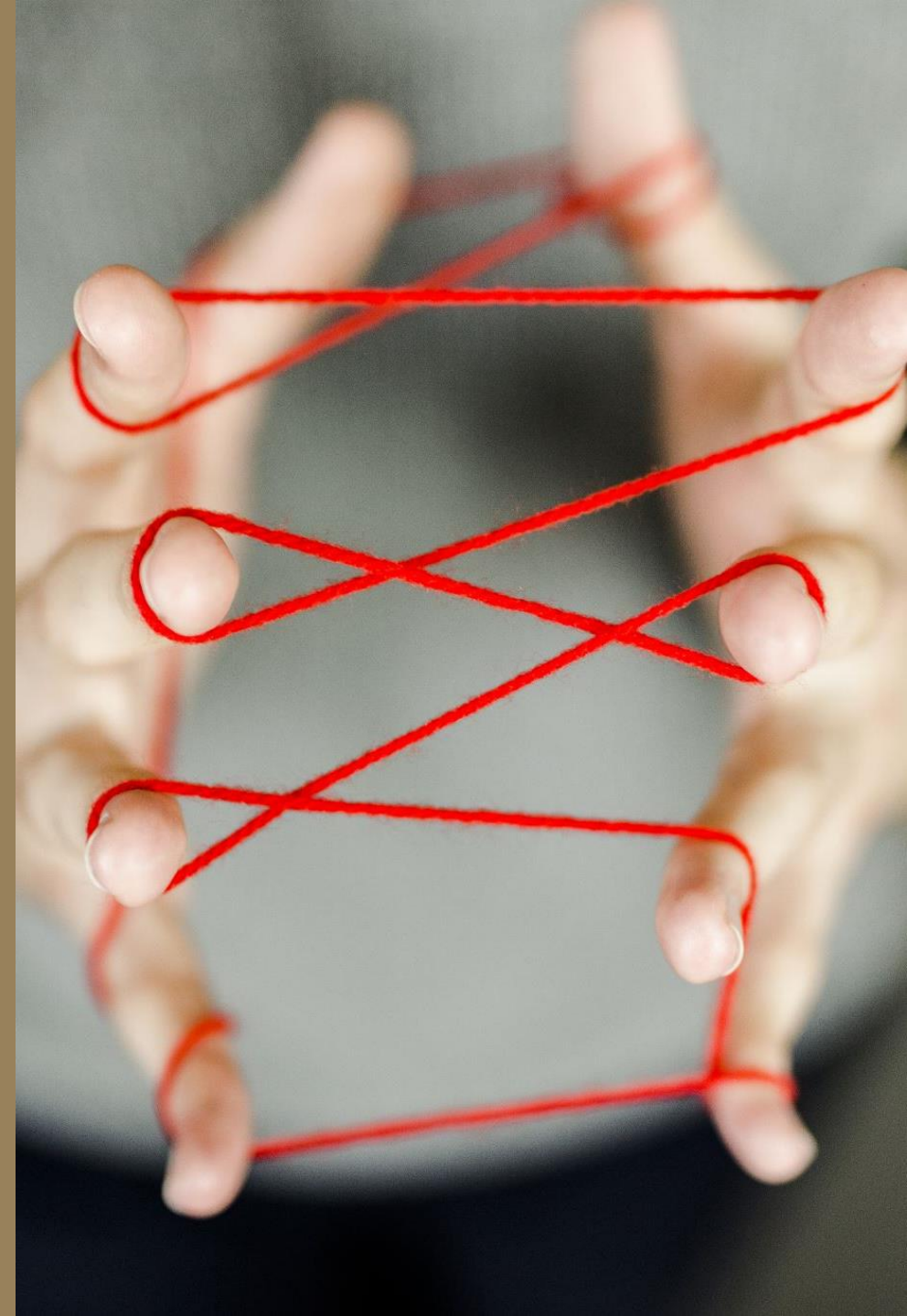
Fully Connected Layers:

1. **Purpose:** Connect every neuron in one layer to every neuron in the next layer, similar to traditional neural networks. These layers are typically used towards the end of the network to make final predictions.



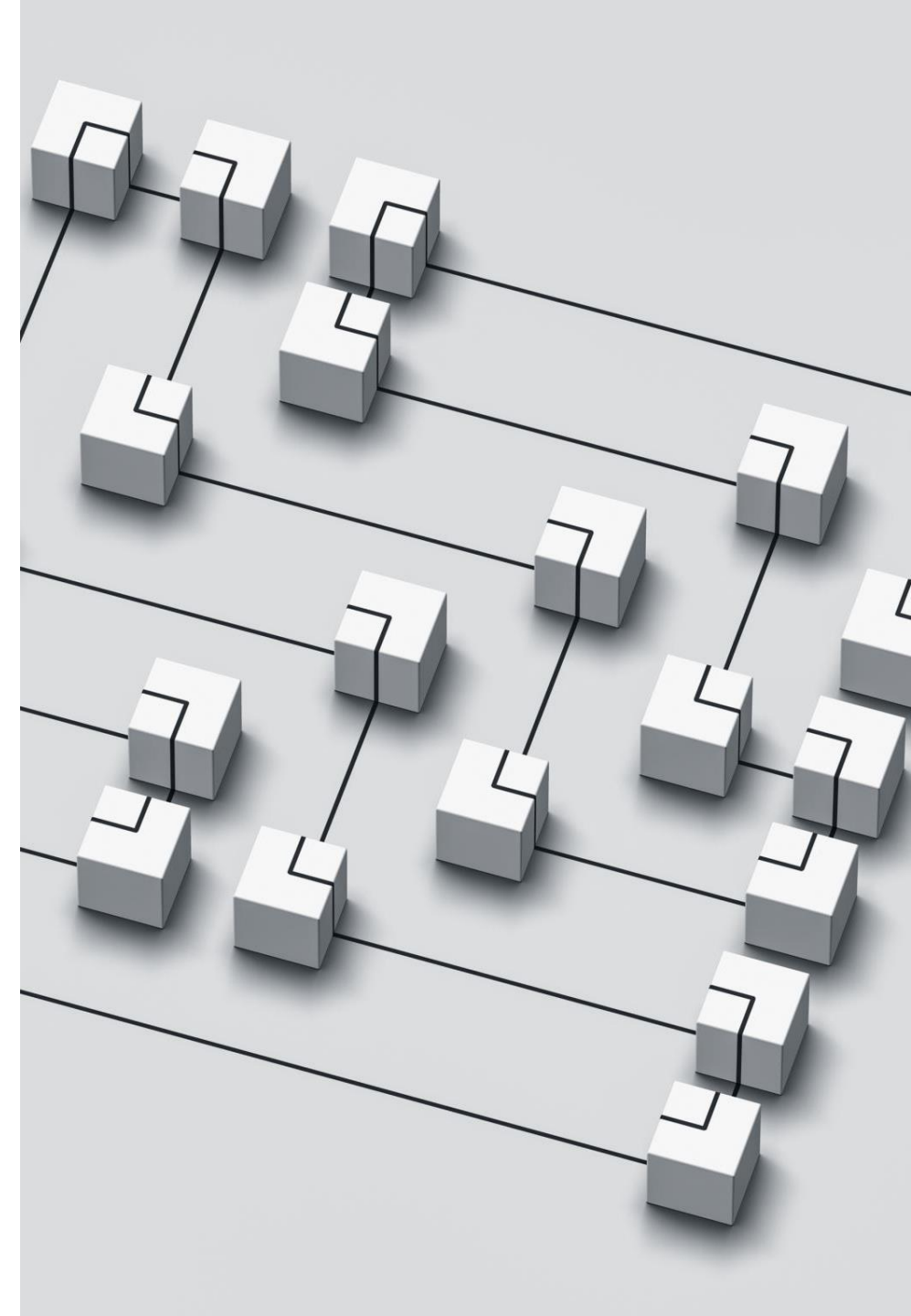
Activation Functions:

1. **Purpose:** Introduce non-linearity into the model, allowing it to learn complex patterns. Common activation functions include ReLU (Rectified Linear Unit), sigmoid, and tanh.



HOW CNNs WORK

- 1. Input Layer:** The raw input data (e.g., an image) is fed into the network.
- 2. Convolution and Pooling:** The input data passes through multiple convolutional and pooling layers to extract hierarchical features.
- 3. Flattening:** The output of the final pooling layer is flattened into a one-dimensional vector.
- 4. Fully Connected Layers:** The flattened vector is passed through one or more fully connected layers.
- 5. Output Layer:** The final layer produces the output, such as class probabilities in a classification task.



APPLICATIONS OF CNNs

- **Image Classification:** Identifying objects within images (e.g., recognizing cats and dogs).
- **Object Detection:** Locating and classifying multiple objects within an image.
- **Image Segmentation:** Dividing an image into segments to identify different objects or regions.
- **Medical Imaging:** Analyzing medical images for diagnosis (e.g., detecting tumors in MRI scans).



RNN VS CNN

https://almablog-media.s3.ap-south-1.amazonaws.com/008_b896ecf606.png

VIDEO

https://www.youtube.com/watch?v=K_BHmztRTpA&t=350s

<https://www.youtube.com/watch?v=IWkFhVq9-nc>

https://www.youtube.com/watch?v=Jy9-aGMB_TE

STEPS TO CREATE NEURAL NETWORKS

1. Define the Problem

Identify the objective: Determine what you want the neural network to achieve (e.g., image classification, regression, etc.).

Gather data: Collect and preprocess the data you'll use to train the network.

2. Design the Neural Network Architecture

Choose the type of neural network: Decide whether to use a feedforward neural network, convolutional neural network (CNN), recurrent neural network (RNN), etc., based on the problem.

Define the layers: Specify the number of layers and the type of each layer (e.g., input layer, hidden layers, output layer).

Select activation functions: Choose activation functions for each layer (e.g., ReLU, sigmoid, tanh).



3. Initialize the Network

Set initial weights: Initialize the weights of the network, often using random values.

Define the loss function: Choose a loss function that measures the difference between the predicted and actual values (e.g., mean squared error, cross-entropy).



4. Train the Network

Forward propagation: Pass the input data through the network to get the output.

Calculate the loss: Use the loss function to compute the error.

Backpropagation: Adjust the weights using an optimization algorithm (e.g., gradient descent) to minimize the loss.

Iterate: Repeat the forward and backward propagation steps for multiple epochs until the network converges.



5. Evaluate the Model

Test on validation data: Use a separate validation dataset to evaluate the performance of the network.

Adjust hyperparameters: Tune hyperparameters (e.g., learning rate, batch size) to improve performance.

6. Deploy the Model

Save the model: Save the trained model for future use.

Integrate into applications: Deploy the model into a production environment where it can make predictions on new data.

TOOLS AND LIBRARIES

Frameworks: Use popular frameworks like TensorFlow, Keras, or PyTorch to build and train your neural network.

Libraries: Utilize libraries for data preprocessing, visualization, and model evaluation.

FEED FORWARD NEURAL NETWORK (FNN)



Architecture: FNNs consist of input layers, hidden layers, and output layers.



Use Cases: FNNs are primarily used for classification and regression



Strengths:

Easy to understand and implement.

Can be used for both classification and regression tasks.



Limitations:

Not well-suited for sequential data or tasks requiring temporal dependencies.

Prone to overfitting, especially with small datasets.

CONVOLUTIONAL NEURAL NETWORKS (CNNs)



Architecture: CNNs consist of convolutional layers, pooling layers, and fully connected layers. The convolutional layers apply filters to the input data to detect features.



Use Cases: CNNs are primarily used for tasks involving spatial data, such as image and video recognition, object detection, and image segmentation.



Strengths:

Excellent at capturing spatial hierarchies in data.
Efficient in handling large images due to parameter sharing and local connectivity.



Limitations:

Not well-suited for sequential data or tasks requiring temporal dependencies.

RECURRENT NEURAL NETWORKS (RNNS)



Architecture: RNNs have a recurrent structure where the output from previous steps is fed back into the network. This allows them to maintain a memory of previous inputs.



Use Cases: RNNs are ideal for tasks involving sequential data, such as natural language processing (NLP), speech recognition, and time series prediction.



Strengths:

Capable of capturing temporal dependencies and patterns in sequential data.
Suitable for tasks where context and order of data are important.



Limitations:

Can suffer from vanishing and exploding gradient problems, making training difficult for long sequences.
Generally slower to train compared to CNNs due to their sequential nature.